Informe de Trabajo Práctico: Motor de Búsqueda de Similitud en Código (AST)

El presente informe detalla el diseño e implementación de un motor de búsqueda capaz de detectar similitudes y posibles duplicados entre funciones escritas en Python. A diferencia de las comparaciones tradicionales de texto plano, que fallan cuando un programador simplemente cambia los espacios o renombra una variable, esta herramienta analiza el esqueleto lógico del código. Para lograrlo, convierte el código fuente en un Árbol de Sintaxis Abstracta (AST) y compara su estructura, imitando el funcionamiento de herramientas profesionales de detección de plagio o calidad de código.

Para cumplir con los requisitos del proyecto, dividimos el sistema en cuatro módulos independientes, asegurando un código ordenado y fácil de mantener:

- Extracción: Se optó por usar la clase NodeVisitor del módulo nativo ast. Esto permite que el programa recorra automáticamente todas las carpetas buscando archivos .py y encuentre los bloques de código que son funciones, ignorando clases completas o código suelto.
- Normalización: Para evitar que los nombres originales afecten la comparación, se implementó un proceso de limpieza. Cada vez que aparece una variable o parámetro nuevo, se lo reemplaza por una etiqueta genérica secuencial (VAR1, VAR2). Los textos y números fijos se transforman en la etiqueta LIT. El nombre de la función original también se oculta. De esta forma, dos funciones idénticas pero con distintos nombres de variables quedan con una estructura idéntica.
- Serialización: Comparar árboles de derivación con múltiples ramas es un proceso computacionalmente complejo. Para optimizarlo, se convierte el árbol en una secuencia lineal de tokens utilizando un recorrido pre-order. Esto implica registrar cada instrucción principal antes de descender a analizar su contenido interno, convirtiendo así la estructura ramificada en una simple lista plana de palabras. Este formato lineal facilita y acelera enormemente la comparación matemática entre las funciones.

Para medir qué tan parecidas son dos funciones, implementamos la Similitud de Jaccard sobre n-gramas.La elegimos debido a que Jaccard utiliza una matemática de conjuntos muy directa: divide la cantidad de bloques de código que ambas funciones tienen en común sobre el total de bloques únicos. Es rápido y efectivo.

En cuanto a los n-gramas decidimos que fueran de tamaño 3. En lugar de comparar palabra por palabra, la secuencia lineal se agrupa en pequeños bloques superpuestos de 3 elementos. Se eligió este tamaño porque permite capturar el contexto de las instrucciones. Si un usuario copia una función pero invierte el orden de dos líneas independientes, la mayoría de los bloques de 3 elementos seguirán siendo idénticos, permitiendo que el sistema detecte la copia.

Definimos un umbral del 80% (0.8) para clasificar a dos funciones como candidatas a duplicado. Un porcentaje menor generaría falsos positivos, por ejemplo, marcando como copia estructuras muy comunes en programación, como un simple ciclo for que suma

números, mientras que exigir un 100% permitiría que copias con mínimas alteraciones pasen desapercibidas.

En cuanto a las limitaciones, aunque nuestra herramienta atrapa los plagios más comunes como los cambios en los nombres de las variables o en los espacios, encontramos algunos tipos de copias que no logramos detectar.

Por un lado, tenemos los clones semánticos: como evaluamos la forma del código y no lo que realmente hace, si dos funciones resuelven el mismo problema pero una usa un ciclo tradicional y la otra usa recursividad, sus árboles serán totalmente distintos y el buscador no notará la copia.

También notamos que si alguien toma un fragmento del código y en su lugar simplemente llama a otra función auxiliar para que resuelva esa parte, la estructura original cambia lo suficiente como para engañar a nuestra fórmula.

Además, al reemplazar todos los textos y números por la etiqueta general LIT, perdimos la capacidad de saber si el código original estaba sumando números o juntando palabras. Por último, descubrimos en las pruebas que en funciones muy cortitas, el simple hecho de invertir el orden de dos líneas independientes altera tanto los bloques de n-gramas que el puntaje matemático cae por debajo del umbral.

Para solucionar estas cuestiones, pensamos en varias mejoras para una versión futura. Podríamos aplicar una limpieza más profunda que detecte estructuras que hacen lo mismo y las unifique antes de comparar, como convertir automáticamente todos los ciclos while en for o acomodar las comparaciones para que siempre se lean igual.

También consideramos agregar un filtro que directamente borre el código que no se ejecuta nunca, como las instrucciones que quedan escritas debajo de un return, para evitar que esas líneas extra arruinen el porcentaje de similitud.

Finalmente, para arreglar la debilidad con las funciones pequeñas que tienen líneas desordenadas, podríamos implementar una regla que ordene alfabéticamente las asignaciones independientes antes de generar la lista de etiquetas, garantizando que el orden visual no afecte a la comparación matemática.